

Homework 4 - Stat 495 - Fall 2025

Daizy Buluma

2025-11-03

Homework 4 - Due Monday, Nov. 3rd by midnight to Gradescope

Practicing Academic Integrity

If you worked with others or used resources outside of provided course material, such as Generative AI (anything besides our textbook, course materials in the repo, labs, R help menu) to help complete this assignment, please acknowledge them below using a bulleted list. Remember that according to our class framework, if you engage with Generative AI tools, the tool and date of use must be listed here, and then a complete record of the chat history must be appended to the assignment.

I acknowledge the following individuals with whom I worked on this assignment:

Name(s) and how they assisted you

-

I used the following sources to help complete this assignment:

Source(s) and where used in your submission

-

PROBLEMS TO TURN IN: Add 1, Add 2, and Add 3

The first two problems use the bbb2 data set from the QSARdata package for Chapter 17 and 18 material. The final problem covers concepts from Chapter 15.

For the applied problems, the response variable of interest is bbb2_Class, which can be found in the bbb2_Outcome data set. We have joined the outcome variable to the QuickProp data set that we want to focus on below. You can read the associated help file in R to learn more about the data set. You saw part of this data set on the exam.

```
data(bbb2)
#?bbb2 # for variable reference and information
mybbb2 <- left_join(bbb2_QuickProp, bbb2_Outcome) %>% dplyr::select(- Molecule)
```

Joining with `by = join\_by(Molecule)`

```
tally(~ bbb2_Class, data = mybbb2)
```

```
bbb2_Class
Crosses DoesNot
      45      35
```

Our goal for the applied problems below is to use the recent methods from class (Chapter 17 and 18) to predict the response variable, whether each compound “crosses” the blood-brain barrier or “does not” cross. You should use the *mybbb2* data set going forward. Note that there are 51 variables at the moment, and the last variable “Class” is the target, but if you open the data set to view, it will only show 50 variables by default. Class is there, but you have to use the arrows to see it.

```
# We loaded a lot of data sets we don't need anymore
# remove them to clean up your workspace
remove(bbb2_AtomPair, bbb2_Daylight_FP, bbb2_Dragon, bbb2_Lcalc, bbb2_moe2D,
        bbb2_moe2D_FP, bbb2_moe3D, bbb2_Outcome, bbb2_PipelinePilot_FP,
        bbb2_QuickProp)
```

To avoid issues with reproducibility, you should set a seed in EACH chunk below where you do a random process, whether that is setting up the train/test split or fitting a model that has some random process involved.

Note that because some of these methods can take a long time to tune, we will NOT be tuning for them. Instead, you will run them using their original packages/functions *ranger*, *gbm*, and

nnet with the inputs specified. This is to introduce you to those different inputs and what they are for/what they do. If you end up applying these models as some part of the final project, be sure you tune the models appropriately! You should use the help function for information about the functions if you are having issues running the models as a starting point.

Additional Problem 1

Your task for this problem is to do some setup, then fit the models described in parts c, d and e, and then compare them in part f. Use all available predictor variables (except what is removed in part a), with no re-expressions. The same training/test split will be used in Additional Problem 2 as well (i.e. you only make this once).

part a. One variable in the data set, `QikProp_.amidine` causes issues with some of these methods. We will remove it here, but can you see why it is problematic? Explain why this variable is not very useful for this analysis.

SOLUTION: The variable `QikProp_.amidine` has almost no variation. Out of 80 observations, 79 are 0 (98.75%) and only 1 is 1 (1.25%). A predictor with essentially only one level provides no useful information for distinguishing between outcomes (near zero-variance variables). Therefore, we exclude this variable before modeling because it will not contribute to predictive performance and may cause issues for some methods.

```
mybbb2 |>
  count(QikProp_.amidine) |>
  mutate(percent = n / sum(n) * 100)
```

	QikProp_.amidine	n	percent
1	0	79	98.75
2	1	1	1.25

```
# run once you are ready to remove the variable to proceed
# this variable is not used anywhere below, so overwrite the data set

mybbb2 <- dplyr::select(mybbb2, -QikProp_.amidine)
```

part b. Create an appropriate training/test split from `mybbb2` with a ratio of 70/30 to use throughout the problems. As always, be sure your split is reproducible.

SOLUTION:

```
set.seed(008)

split <- initial_split(mybbb2, prop = 0.70, strata = "Class")
cross_train <- training(split)
cross_test <- testing(split)

#tally(~ Class, data = cross_train, format = "count")
```

part c. Create an appropriate model to predict Class with the training set using bagging with 1000 trees and otherwise with default settings for tuning parameter values.

SOLUTION:

```
set.seed(008)
bag_cross <- ranger(
  formula = Class ~ .,
  data = cross_train,
  mtry = 49, #number of predictors
  num.trees = 1000
)
#bag_cross
```

part d. Create an appropriate model to predict Class with the training set using a random forest with 1000 trees and otherwise with default settings for tuning parameter values.

SOLUTION:

```
set.seed(008)
rf_cross <- ranger(
  formula = Class ~ .,
  data = cross_train,
  num.trees = 1000
)
#rf_cross
```

part e. Create an appropriate model to predict Class with the training set using boosting with 500 trees and otherwise with default settings for tuning parameter values.

SOLUTION:

```
cross_train$Classnum <- ifelse(cross_train$Class == "Crosses", 1, 0)
cross_test$Classnum <- ifelse(cross_test$Class == "Crosses", 1, 0)

set.seed(008)
boost_cross <- gbm(
  formula = Classnum ~ .,
  data = cross_train|> dplyr::select(-Class),
  n.trees = 500,
```

```
distribution = "bernoulli")
#boost_cross
```

part f. Parts c, d, and e only required you to fit models. Now we want to compare their performance. Use an appropriate measure to compare the three models in terms of their model performance based on the test set. Be sure your choice of measure is clear. Summarize your findings. Then discuss which model you would choose for predicting *Class*. Explain your choice. (You can use additional assessment items, but must have the test set comparison.)

SOLUTION:

I used the estimated True Error Rate (TER) to compare the performance of the three models. The bagging and boosting models both achieved about 84% test accuracy, giving an estimated TER of about 0.16. The random forest model had slightly lower test accuracy (around 80%), leading to an estimated TER of about 0.20. Based on this test set comparison, bagging and boosting appear to perform the best.

Although bagging and boosting had the same estimated TER, I would choose the bagging model for predicting *Class* because it produced strong predictive performance and was simpler to implement and evaluate. Also, boosting generally requires more tuning than bagging and can easily overfit if the hyperparameters are not carefully selected.

```
#for bagging - 84%
bag_test_pred <- predict(bag_cross, cross_test)$predictions
confusionMatrix(bag_test_pred , cross_test$Class)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Crosses	DoesNot
Crosses	11	1
DoesNot	3	10

Accuracy : 0.84
 95% CI : (0.639, 0.955)
 No Information Rate : 0.56
 P-Value [Acc > NIR] : 0.00311

 Kappa : 0.682

 McNemar's Test P-Value : 0.61708

```

        Sensitivity : 0.786
        Specificity : 0.909
        Pos Pred Value : 0.917
        Neg Pred Value : 0.769
        Prevalence : 0.560
        Detection Rate : 0.440
        Detection Prevalence : 0.480
        Balanced Accuracy : 0.847

        'Positive' Class : Crosses

```

```

#for random forest - 80%
rf_test_pred <- predict(rf_cross, cross_test)$predictions
confusionMatrix(rf_test_pred , cross_test$Class)

```

Confusion Matrix and Statistics

```

              Reference
Prediction Crosses DoesNot
Crosses      11         2
DoesNot      3         9

        Accuracy : 0.8
        95% CI : (0.593, 0.932)
        No Information Rate : 0.56
        P-Value [Acc > NIR] : 0.0112

        Kappa : 0.598

McNemar's Test P-Value : 1.0000

```

```

        Sensitivity : 0.786
        Specificity : 0.818
        Pos Pred Value : 0.846
        Neg Pred Value : 0.750
        Prevalence : 0.560
        Detection Rate : 0.440
        Detection Prevalence : 0.520
        Balanced Accuracy : 0.802

        'Positive' Class : Crosses

```

```

#for boosting - 84%
boost_data_test <- cross_test |> dplyr::select(-Class)
boost_test_pred <- predict(boost_cross,
                           newdata = boost_data_test,
                           n.trees = 500,
                           type = "response")

# Convert predicted probabilities to class labels
boost_class <- factor(ifelse(boost_test_pred > 0.5, 1, 0),
                      levels = c(0,1),
                      labels = c("DoesNot", "Crosses"))

# Ensure same factor format as predicted
boost_data_test$Classnum <- factor(boost_data_test$Classnum,
                                   levels = c(0,1),
                                   labels = c("DoesNot", "Crosses"))

confusionMatrix(boost_class, boost_data_test$Classnum, positive = "Crosses")

```

Confusion Matrix and Statistics

	Reference	
Prediction	DoesNot	Crosses
DoesNot	10	3
Crosses	1	11

Accuracy : 0.84
 95% CI : (0.639, 0.955)
 No Information Rate : 0.56
 P-Value [Acc > NIR] : 0.00311

 Kappa : 0.682

 Mcnemar's Test P-Value : 0.61708

 Sensitivity : 0.786
 Specificity : 0.909
 Pos Pred Value : 0.917
 Neg Pred Value : 0.769
 Prevalence : 0.560

Detection Rate : 0.440
Detection Prevalence : 0.480
Balanced Accuracy : 0.847

'Positive' Class : Crosses

Additional Problem 2

Your task for this problem is to fit the models described in parts a, b, and c, and then compare them in part d. Use all available predictor variables, with no re-expressions. Use the same training/test data as above. Finally, in part e, you'll try to do better with a model of your own.

(Remember, these tuning parameter values were arbitrary choices I made so you can see the inputs and such to the models. They are not tuned for optimal performance!)

part a. Create an appropriate model to predict Class with the training set using a neural net with a single hidden layer of 12 nodes, and otherwise with default settings for tuning parameter values.

SOLUTION:

```
set.seed(008)
nnet_cross <- nnet(Class ~ ., cross_train, size = 12)
```

```
# weights:  613
initial  value 31.518766
iter   10 value 14.736944
iter   20 value 12.278440
iter   30 value 11.043492
iter   40 value  9.803099
iter   50 value  9.778549
final   value  9.778410
converged
```

part b. Create an appropriate model to predict Class with the training set using a neural net with a single hidden layer of 12 nodes, and a decay parameter of $6e-3$, and otherwise with default settings for tuning parameter values.

SOLUTION:

```
set.seed(008)
nnet_cross2 <- nnet(Class ~ ., cross_train, size = 12, decay = 6e-3)
```

```
# weights:  613
initial  value 32.116749
iter   10 value 15.001993
iter   20 value 14.253845
iter   30 value 12.844181
```

```

iter  40 value 11.816513
iter  50 value  9.535175
iter  60 value  9.431950
iter  70 value  9.291734
iter  80 value  8.982963
iter  90 value  7.978277
iter 100 value  6.678349
final  value  6.678349
stopped after 100 iterations

```

part c. Create an appropriate model to predict Class with the training set using a neural net with a single hidden layer of 12 nodes, a decay parameter of $6e-3$, and a value for maxit that allows for convergence, and otherwise with default settings for tuning parameter values. (Ask if you aren't sure what the convergence idea is about!)

SOLUTION:

```

set.seed(008)
nnet_cross3 <- nnet(
  Class ~ .,
  data = cross_train,
  size = 12,
  decay = 6e-3,
  maxit = 630)

```

```

# weights:  613
initial  value 32.116749
iter  10 value 15.001993
iter  20 value 14.253845
iter  30 value 12.844181
iter  40 value 11.816513
iter  50 value  9.535175
iter  60 value  9.431950
iter  70 value  9.291734
iter  80 value  8.982963
iter  90 value  7.978277
iter 100 value  6.678349
iter 110 value  6.017563
iter 120 value  5.140771
iter 130 value  5.070511
iter 140 value  4.866885
iter 150 value  4.728762

```

```
iter 160 value 4.706859
iter 170 value 4.657582
iter 180 value 4.274487
iter 190 value 4.208966
iter 200 value 4.201336
iter 210 value 3.880525
iter 220 value 3.676499
iter 230 value 3.391007
iter 240 value 3.174601
iter 250 value 3.086791
iter 260 value 3.059608
iter 270 value 3.030384
iter 280 value 3.010304
iter 290 value 3.001312
iter 300 value 2.978686
iter 310 value 2.968643
iter 320 value 2.810396
iter 330 value 2.043143
iter 340 value 1.666900
iter 350 value 1.639661
iter 360 value 1.625974
iter 370 value 1.560913
iter 380 value 1.514707
iter 390 value 1.500762
iter 400 value 1.484109
iter 410 value 1.450108
iter 420 value 1.406812
iter 430 value 1.372612
iter 440 value 1.324200
iter 450 value 1.217451
iter 460 value 1.141746
iter 470 value 1.110015
iter 480 value 1.104038
iter 490 value 1.091379
iter 500 value 1.085435
iter 510 value 1.051961
iter 520 value 1.003112
iter 530 value 0.979153
iter 540 value 0.971555
iter 550 value 0.968467
iter 560 value 0.967143
iter 570 value 0.966746
iter 580 value 0.966675
```

```

iter 590 value 0.966569
iter 600 value 0.966481
iter 610 value 0.966468
iter 620 value 0.966467
final value 0.966467
converged

```

part d. Parts a, b, and c only required you to fit models. Now we want to compare their performance. Use an appropriate measure to compare the three models in terms of their model performance based on the test set. Be sure your choice of measure is clear. Summarize your findings. Then discuss which model you would choose for predicting Class. Explain your choice.

SOLUTION:

I used test set accuracy as the performance measure. Based on the confusion matrices, the `nnet_cross` achieved a test accuracy of about 0.80, which corresponds to an estimated TER of 0.20. `nnet_cross2` achieved a test accuracy of about 0.76 and an estimated TER of 0.24. Finally, `nnet_cross3`, also achieved a test accuracy of 0.80 and therefore the same estimated TER of 0.20.

`nnet_cross` and `nnet_cross3` both converge and achieve the same accuracy on the test set. However `nnet_cross3` includes weight decay, which shrinks weights and helps guard against overfitting and typically yields more stable performance across splits. I would therefore choose `nnet_cross3`.

```

# nnet_cross
pred1 <- predict(nnet_cross, newdata = cross_test, type = "class")
pred1 <- factor(pred1, levels = levels(cross_test$Class))
confusionMatrix(pred1, cross_test$Class)

```

Confusion Matrix and Statistics

	Reference	
Prediction	Crosses	DoesNot
Crosses	11	2
DoesNot	3	9

Accuracy : 0.8
 95% CI : (0.593, 0.932)
 No Information Rate : 0.56
 P-Value [Acc > NIR] : 0.0112

 Kappa : 0.598

McNemar's Test P-Value : 1.0000

Sensitivity : 0.786
Specificity : 0.818
Pos Pred Value : 0.846
Neg Pred Value : 0.750
Prevalence : 0.560
Detection Rate : 0.440
Detection Prevalence : 0.520
Balanced Accuracy : 0.802

'Positive' Class : Crosses

```
# nnet_cross2  
pred2 <- predict(nnet_cross2, newdata = cross_test, type = "class")  
pred2 <- factor(pred2, levels = levels(cross_test$Class))  
confusionMatrix(pred2, cross_test$Class)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Crosses	DoesNot
Crosses	11	3
DoesNot	3	8

Accuracy : 0.76
95% CI : (0.549, 0.906)
No Information Rate : 0.56
P-Value [Acc > NIR] : 0.0323

Kappa : 0.513

McNemar's Test P-Value : 1.0000

Sensitivity : 0.786
Specificity : 0.727
Pos Pred Value : 0.786
Neg Pred Value : 0.727
Prevalence : 0.560
Detection Rate : 0.440

Detection Prevalence : 0.560
Balanced Accuracy : 0.756

'Positive' Class : Crosses

```
# nnet_cross3
pred3 <- predict(nnet_cross3, newdata = cross_test, type = "class")
pred3 <- factor(pred3, levels = levels(cross_test$Class))
confusionMatrix(pred3, cross_test$Class)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Crosses	DoesNot
Crosses	11	2
DoesNot	3	9

Accuracy : 0.8
95% CI : (0.593, 0.932)
No Information Rate : 0.56
P-Value [Acc > NIR] : 0.0112

Kappa : 0.598

McNemar's Test P-Value : 1.0000

Sensitivity : 0.786
Specificity : 0.818
Pos Pred Value : 0.846
Neg Pred Value : 0.750
Prevalence : 0.560
Detection Rate : 0.440
Detection Prevalence : 0.520
Balanced Accuracy : 0.802

'Positive' Class : Crosses

part e. Fit a new neural net model with parameters chosen by you (you can even explore options we didn't use above), but must set the size of the hidden layer and ensure your model reaches convergence. What parameter values did you use? Does your model do better than your final model in part d.?

SOLUTION: I used `size = 10`, `decay = 0.1`, and `maxit = 1000`. The model converged but did not outperform the best model from part (d).

```
set.seed(008)
nnet_cross4 <- nnet(
  Class ~ .,
  data = cross_train,
  size = 10,
  decay = 0.1,
  maxit = 1000)
```

```
# weights:  511
initial  value 40.201585
iter  10 value 26.384843
iter  20 value 22.671069
iter  30 value 15.312886
iter  40 value 14.823737
iter  50 value 14.222787
iter  60 value 13.616146
iter  70 value 12.210318
iter  80 value 11.516679
iter  90 value 11.117036
iter 100 value 10.598248
iter 110 value 10.000813
iter 120 value  9.660664
iter 130 value  9.353808
iter 140 value  8.754359
iter 150 value  7.944222
iter 160 value  7.669470
iter 170 value  7.192712
iter 180 value  6.608206
iter 190 value  6.388224
iter 200 value  6.306903
iter 210 value  6.265175
iter 220 value  6.252412
iter 230 value  6.248408
iter 240 value  6.246723
iter 250 value  6.245481
iter 260 value  6.219451
iter 270 value  6.184422
iter 280 value  5.977831
iter 290 value  5.766730
```



```

iter 300 value 5.560267
iter 310 value 5.437337
iter 320 value 5.369294
iter 330 value 5.355389
iter 340 value 5.308148
iter 350 value 5.288842
iter 360 value 5.288336
iter 370 value 5.288227
iter 380 value 5.288220
iter 390 value 5.288217
iter 400 value 5.288216
iter 400 value 5.288216
iter 400 value 5.288216
final value 5.288216
converged

```

```

pred4 <- predict(nnet_cross4, newdata = cross_test, type = "class")

# Match factor levels
pred4 <- factor(pred4, levels = levels(cross_test$Class))

confusionMatrix(pred4, cross_test$Class, positive = "Crosses")

```

Confusion Matrix and Statistics

	Reference	
Prediction	Crosses	DoesNot
Crosses	11	2
DoesNot	3	9

Accuracy : 0.8
 95% CI : (0.593, 0.932)
 No Information Rate : 0.56
 P-Value [Acc > NIR] : 0.0112

 Kappa : 0.598

 McNemar's Test P-Value : 1.0000

 Sensitivity : 0.786
 Specificity : 0.818
 Pos Pred Value : 0.846

Neg Pred Value : 0.750
Prevalence : 0.560
Detection Rate : 0.440
Detection Prevalence : 0.520
Balanced Accuracy : 0.802

'Positive' Class : Crosses

Additional Problem 3

In a few sentences, explain to someone currently taking STAT 135 (so their level of background knowledge is intro stats, just having seen hypothesis testing for the first time), what p-value adjustment is all about and why it's an important concept to understand.

SOLUTION:

As you know, when performing a hypothesis test, we usually calculate a p-value to help us decide whether our result is statistically significant or whether occurred just by random chance. We compare this p-value against a threshold, alpha, which represents the probability of making a Type I error (incorrectly concluding there's an effect when there really isn't one). Now suppose we set $\alpha = 0.05$ and test 100 different variables to see if any are related to an outcome. Even if none of them are truly related, we'd still expect about five of them to show a "significant" result just by chance because 0.05 means there's a 5% chance of a false positive for each test.

When you run multiple tests, each one carries its own 5% risk. Those risks start to add up and the overall chance of making at least one false positive across all tests becomes much higher than 5%. This where p-value adjustment comes in. It helps bring that overall risk back down to a reasonable level. This is done by making the criteria for significance stricter when performing multiple comparisons. There are several adjustment methods, but they all aim to reduce the likelihood of mistakenly claiming a discovery when the result is really just noise

Understanding p-value adjustment is important because studies often involve multiple hypothesis tests at once. Without it, we might believe we've found something meaningful when, in reality, the result occurred by random chance. The adjustment gives you more confidence that the results you call "significant" are actually meaningful.